



Name: Hamna Munib, Mahad khan	Student ID: hm09929, mk10570	Section: T3
-------------------------------	---------------------------------	-------------

Lab 2: Implementing Decision Instructions in RISC V Assembly Language

Task 1:

Type the codes in Listings 3 and 4 in the assembler editor and then in the simulator look at their binary:

- The beq and bne instructions are stored in the SB format with their 32-bit binary. Look it up and try to find out what values their different fields contain.
- Looking at the binary formats, find out how do the labels are stored in the 32-bit instruction. Compare the label values stored inside the instruction to the code and try to guess how the assembler stores the jump addresses inside the instruction. (Hint: it uses a PC-relative scheme) Refer to following figures (from book page 119) to see encoding of instructions.

Note: To test listing 4, ld will be replaced by lw and for computation of address, offset should be obtained by $i*4$ i.e., shift left by 2 instead of

For Listing 3

```
1  if (i == j)
2      f = g + h;
3  else
4      f = g - h;
5  // code after if/else goes here
6
7
8
9  //assuming that variables f to j are in registers x19-x23
10 bne x22, x23, Else
11 add x19, x20, x21
12 beq x0, x0, Exit //un conditional jump
13 Else: sub x19, x20, x21
14
15 Exit: # the code after if/else goes here
```

Listing 3: If statement

BEQ

0000000	00000	00000	000	10000	1100011
---------	-------	-------	-----	-------	---------

BNE

0000000	10111	10110	001	11000	1100011
---------	-------	-------	-----	-------	---------



For listing 4

```
1  while (save[i] == k)
2      i += 1;
3
4
5  // assuming i and k in x22 and x24, and the base address of Save in x25
6  Loop: slli x10, x22, 3 // Temp reg x10 = i * 8
7        add x10, x10, x25 // x10 = address of save[i]
8        ld x9, 0(x10) // Temp reg x9 = save[i]
9        bne x9, x24, Exit // go to Exit if save[i] != k
10       addi x22, x22, 1 // i = i + 1
11       beq x0, x0, Loop // go to Loop
12  Exit:
13
```

Listing 4: while loop

BEQ

1111110	00000	00000	000	11001	1100011
---------	-------	-------	-----	-------	---------

BNE

0000000	11000	01001	001	11000	1100011
---------	-------	-------	-----	-------	---------

Task 2:

The switch statement is similar to if/else statements. Write the equivalent RISC-V assembly code for Listing 5. Assume that the variables x, a, b & c are signed integers and stored in x20, x21, x22, x23 respectively. Make sure to first assign suitable values for b and c first.

```
1 switch (x) {
2     case 1:
3         a = b+c;
4         break;
5     case 2:
6         a = b-c;
7         break;
8     case 3:
9         a = b * 2;
10        break;
11    case 4:
12        a = b / 2;
13        break;
14    default:
15        a = 0;
16 }
```

Listing 5: task 1b.c

Code:

```
main:
    li x1, 1 # for case 1
    li x2, 2 # for case 2
    li x3, 3 # for case 3
    li x4, 4 # for case 4

    #sample values for a,b,c,d
    li x20, 1 # x
    li x21, 1 # a
    li x22, 2 # b
    li x23, 3 # c
    beq x1, x20, case1
    beq x2, x20, case2
    beq x3, x20, case3
    beq x4, x20, case4
    beq x0, x0, default

case1:
    add x21, x22, x23 # a = b+c
    j Exit

case2: sub x21, x22, x23 #a = b-c
    j Exit

case3: slli x21, x22, 1 #a = b*2
    j Exit

case4: srli x21, x22, 1 #a = b/2
    j Exit

default:
    li x20, 0 # a= 0
    j Exit

Exit:
    j Exit
```

Output:

case 1 (when x=1 in register x20)
final Value of a in register x21 = 5

x20 (s4)	=	0x00000001
x21 (s5)	=	0x00000005
x22 (s6)	=	0x00000002
x23 (s7)	=	0x00000003

case 2 (when x=2 in register x20)
final Value of a in register x21 = -1
a = b - c
a = 2-3
a = -1 (0xffffffff in hex)

x20 (s4)	=	0x00000002
x21 (s5)	=	0xFFFFFFFF
x22 (s6)	=	0x00000002
x23 (s7)	=	0x00000003

case 3 (when x=3 in register x20)
final Value of a in register x21 =4
a = b*2 = 2*2 = 4

x20 (s4)	=	0x00000003
x21 (s5)	=	0x00000004
x22 (s6)	=	0x00000002
x23 (s7)	=	0x00000003

case 4 (when x=4 in register x20)
final Value of a in register x21 =1
a = b/2 = 2/2 = 1

x20 (s4)	=	0x00000004
x21 (s5)	=	0x00000001
x22 (s6)	=	0x00000002
x23 (s7)	=	0x00000003

default case
final Value of a in register x21 = 0

x20 (s4)	=	0x00000008
x21 (s5)	=	0x00000000
x22 (s6)	=	0x00000002
x23 (s7)	=	0x00000003

Task 3:

Write the equivalent RISC-V assembly code for Listing 6. Assume that the variables `i` and `sum` are in `x22` and `x23`, while the array `a` located at address `0x200` is of 4-byte integers.

```
1 for (int i=0; i<10; i++)
2   a[i] = i;
3
4 for (int i=0; i<10; i++)
5   sum = sum+a[i];
```

Listing 6: Reduction Sum

Code:

```
ASM task3.s
1  .text
2  .globl main
3  main:
4      li x5, 0x200 # base adress of a
5      li x22, 0 # i = 0
6      li x23, 0 # sum
7      li x24, 10
8
9  L1:
10     beq x22, x24, END1
11     j loop1
12  loop1:
13     slli x6, x22, 2
14     add x7, x5, x6
15     sw x22, 0(x7)
16     addi x22, x22, 1
17     j L1
18  END1:
19     li x22, 0 # reset i = 0
20
21  L2:
22     beq x22, x24, END2
23     j loop2
24  loop2:
25     slli x6, x22, 2
26     add x7, x5, x6
27     lw x6, 0(x7)
28     add x23, x23, x6
29     addi x22, x22, 1
30     j L2
31  END2:
32
33  end:
34     j end
35
```



Output:

Final sum in register x23 - 2D (in hex) which is 45 in binary

x22 (s6)	= 0x0000000A
x23 (s7)	= 0x0000002D
x24 (s8)	= 0x0000000A

Value of i stored in array a (base address x200)

Address	+0	+1	+2	+3
0x00000230	00	00	00	00
0x0000022C	00	00	00	00
0x00000228	00	00	00	00
0x00000224	09	00	00	00
0x00000220	08	00	00	00
0x0000021C	07	00	00	00
0x00000218	06	00	00	00
0x00000214	05	00	00	00
0x00000210	04	00	00	00
0x0000020C	03	00	00	00
0x00000208	02	00	00	00
0x00000204	01	00	00	00
0x00000200	00	00	00	00

Address: Up Down

Jump to: -- choose --

Display Format: Hex

Bytes per Row: 4

Task 4 (Challenge):

Translate the following C code to RISC-V assembly code. Use a minimum number of instructions. Assume that the values of a, b, i, and j are in registers x5, x6, x7, and x29, respectively. Also, assume that register x10 holds the base address of the array D.

```
1
2 for (i=0; i<a; i++)
3     for (j=0; j<b; j++)
4         D[4*j] = i + j;
```

Listing 7: Nested Loops

Code:

ASM task4.s

```
1  .text
2  .globl
3  main:
4  li x5, 5 # a = 5
5  li x6, 5 # b = 5
6  li x7, 0 # i = 0
7  li x10, 0x200 # base address of D
8
9  C1:
10     beq x7, x5, end
11     li x29, 0 # j = 0
12
13  C2:
14     beq x29, x6, C3
15     j innerloop
16
17  innerloop:
18     add x20, x7, x29 # storing j + i into temp register
19     slli x21, x29, 2 # storing j*4 into temp register
20     add x21, x21, x10
21     sb x20, 0(x21)
22     addi x29, x29, 1 # j = j + 1
23     j C2
24
25  C3:
26     addi x7, x7, 1 # i = i + 1
27     j C1
28
29  end:
30     j end
31
```

Output

Final value of array D (base address x200)

Memory

Address	+0	+1	+2	+3
0x00000230	00	00	00	00
0x0000022C	00	00	00	00
0x00000228	00	00	00	00
0x00000224	00	00	00	00
0x00000220	00	00	00	00
0x0000021C	00	00	00	00
0x00000218	00	00	00	00
0x00000214	00	00	00	00
0x00000210	08	00	00	00
0x0000020C	07	00	00	00
0x00000208	06	00	00	00
0x00000204	05	00	00	00
0x00000200	04	00	00	00

Address:
Up
Down

Jump to: -- choose --

Display Format: Hex

Bytes per Row: 4

```

x06 (t1) = 0x00000005
x07 (t2) = 0x00000005 —> Final value of i
x08 (s0) = 0x00000000
x09 (s1) = 0x00000000
x10 (a0) = 0x00000200 —> base address of array D

```

```

x20 (s4) = 0x00000008 —> final value of i + j = 4 + 4 = 8
x21 (s5) = 0x00000210 —> last index of D
x22 (s6) = 0x00000000
x23 (s7) = 0x00000000
x24 (s8) = 0x00000000
x25 (s9) = 0x00000000
x26 (s10) = 0x00000000
x27 (s11) = 0x00000000
x28 (t3) = 0x00000000
x29 (t4) = 0x00000005 —> Final value of j

```




Assessment Rubric

Lab 2: Implementing Decision Instructions in RISC V Assembly Language

Name:	Student ID:	Section:
--------------	--------------------	-----------------

Points Distribution

	Task No.	LR 2 Code	LR 5 Results
In - Lab	Task 1	/0	/5
	Task 2	/10	/10
	Task 3	/15	/10
	Task 4	/20	/10
Total Points: 100		/45	/35
CLO Mapped		CLO2	

Affective Domain Rubric		Points	CLO
AR7	Report	/20	CLO 2

CLO	Total Points	Points Obtained
2	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.